Quickselect

AlgoExpert · Hard · Searching

The problem

- Kth smallest element (1-based) of an unsorted array, in place with Hoare-style partitioning.

Intuition

- Partition like quicksort, but only recurse into the side containing the kth position.

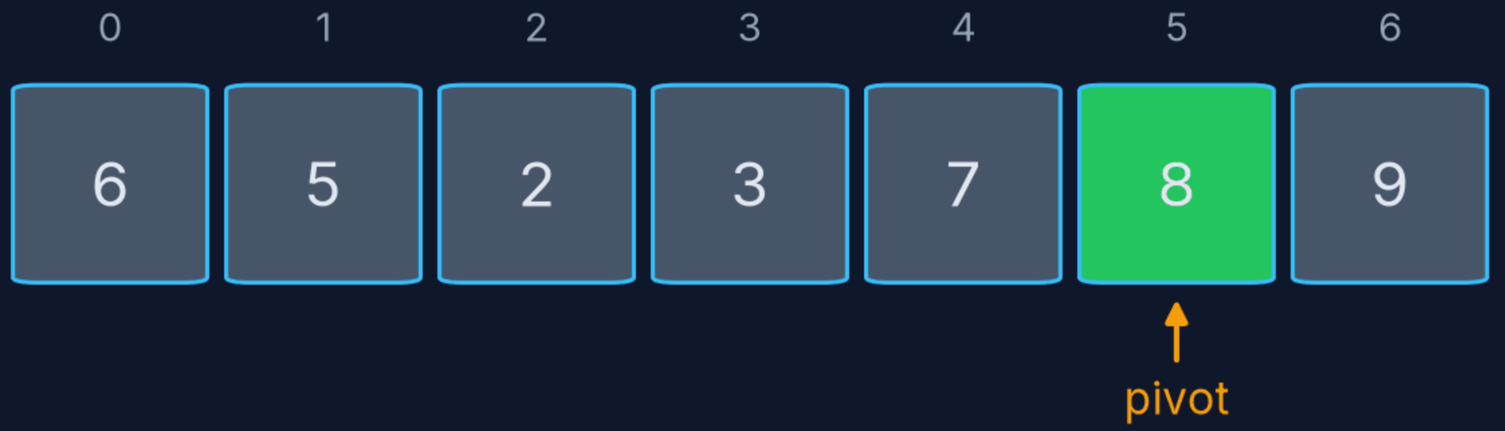
Brute force vs optimal

- Sort → O(n log n)
- Heap of size k → O(n log k)
- Quickselect → O(n) average

Approach

- 1. Pick a pivot and partition
- 2. If the pivot lands at k minus one, done
- 3. Else continue on one side

Worked example




```
pivot 8 lands at index 5; target index 2
```

Pivot 8 → index 5

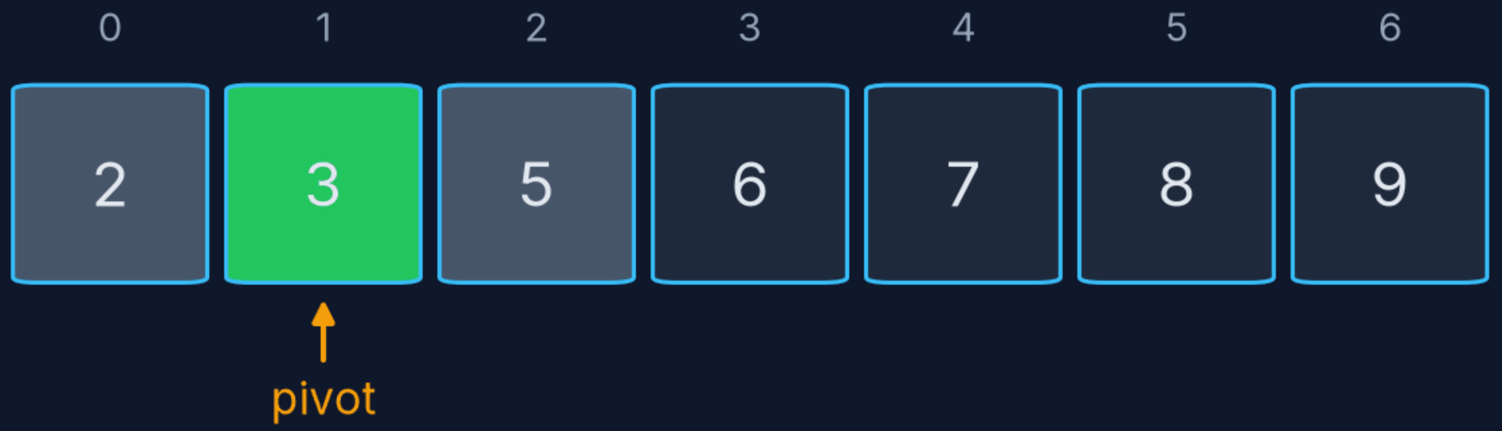
Worked example



`pivot 6 lands at index 3; target index 2`

Pivot 6 → index 3

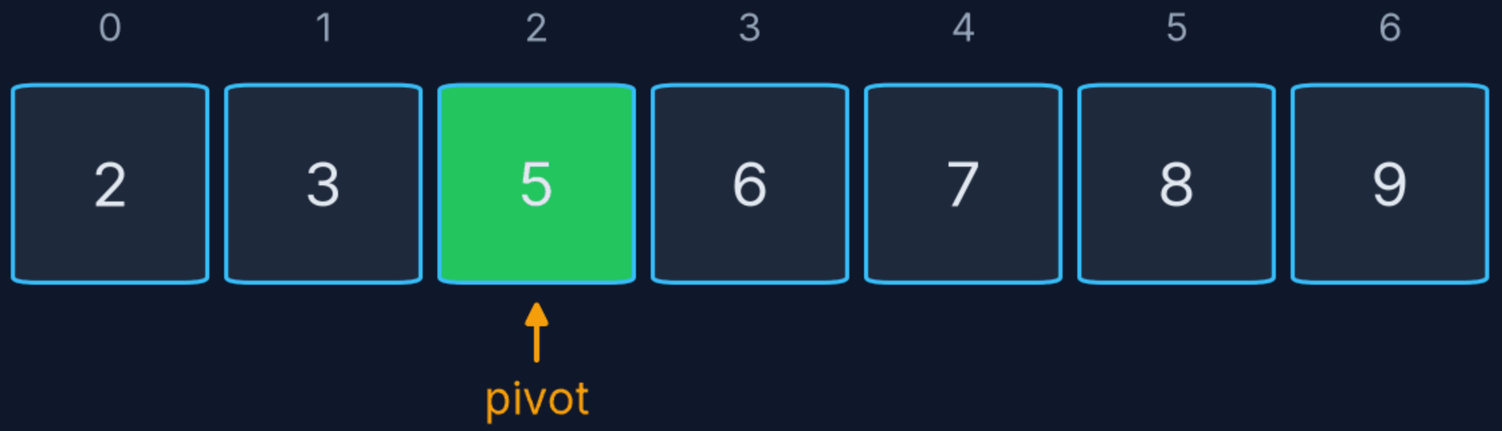
Worked example



`pivot 3 lands at index 1; target index 2`

Pivot 3 → index 1

Worked example



`pivot 5 lands at index 2; target index 2`

Pivot 5 → index 2

Java solution

```java
public class Quickselect {
    public static int quickselect(int[] a, int k) {
        int lo = 0, hi = a.length - 1, target = k - 1;
        while (true) {
            int p = lo, l = lo + 1, r = hi;
            while (l <= r) {
                if (a[l] > a[p] && a[r] < a[p]) swap(a, l, r);
                if (a[l] <= a[p]) l++;
                if (a[r] >= a[p]) r--;
            }
            swap(a, p, r);
            if (r == target) return a[r];
            if (r < target) lo = r + 1; else hi = r - 1;
        }
    }

    static void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }
}
```

Complexity

- Time: O(n) avg, O(n^2) worst
- Space: O(1)

Edge cases & pitfalls

- Worst case $O(n^2)$ — randomise the pivot
- K is 1-indexed
- Duplicates

Interview tips & follow-ups

- Mention median-of-medians O(n) worst case

Recap

- Partition like quicksort, but only recurse into the side containing the kth position.
- Complexity: O(n) avg, O(n^2) worst